

Consultas a una o varias tablas

Módulo: Fundamentos de bases de datos relacionales

| **AE2:** Utilizar lenguaje estructurado de consultas SQL para la obtención de información que satisface los requerimientos planteados a partir de un modelo de datos dado

Introducción

Durante esta lección definiremos y analizaremos las bases de datos relacionales también conocidas como **SQL (Structured Query Language)**, que permite a las organizaciones interactuar con las bases de datos realizando diversas operaciones, como consultar, insertar, actualizar y eliminar datos y obtener información necesaria para sus objetivos de negocio.

Explicaremos el concepto modelo de datos que ayudan a garantizar la integridad y coherencia de los datos, y proporcionan una base para el diseño y desarrollo de bases de datos, profundizaremos sobre **la integridad referencial** para establecer relaciones entre las tablas y, por último, abordaremos las **consultas anidadas** permiten realizar una consulta dentro de otra, utilizando SQL ya que ofrece funciones avanzadas de recuperación y manipulación de datos.

En definitiva, aprenderemos a realizar cláusulas SQL (SELECT, FROM, WHERE y HAVING, etc.) para hacer diferentes consultas que permiten manipular datos complejos, realizar filtrados dinámicos y unir datos de varias tablas.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Definir el Lenguaje Estructurado de Consultas SQL
- Recuperar información de una tabla utilizando consultas SQL
- Conocer el concepto de claves primarias y foráneas
- Conocer el concepto de Modelo de Datos
- Conocer el concepto de Integridad referencial
- Realizar consultas SQL utilizando claves primarias y foráneas
- Hacer consultas anidadas utilizando SQL

El Lenguaje Estructurado de Consultas (SQL)

SQL (Structured Query Language) es el lenguaje estándar utilizado para gestionar y manipular datos en bases de datos relacionales, permitiendo realizar consultas eficientes y modificaciones estructurales de manera precisa. Gracias al uso de álgebra y cálculo relacional, SQL facilita la obtención de información y el control sobre grandes volúmenes de datos, lo que lo convierte en una herramienta indispensable en el ámbito de la tecnología. En un mundo donde los datos impulsan decisiones y experiencias digitales, comprender SQL es esencial para cualquier profesional de IT.

El conocimiento de SQL permite crear productos digitales dinámicos y funcionales. Por ejemplo, en una **plataforma de e-commerce**, SQL se utiliza para consultar inventarios, gestionar pedidos y personalizar recomendaciones de productos según el historial de compras del usuario. Este manejo eficiente de datos posibilita experiencias personalizadas y optimizadas. Además, dominar SQL es un paso crucial para desarrollar aplicaciones robustas, diseñar sistemas de información escalables y aprovechar el valor estratégico de los datos en cualquier proyecto digital.

Pero podemos decir, en realidad, que SQL no se usa solamente para la manipulación y consulta de datos, sino que también puede utilizarse para crear y modificar objetos de una base de datos, como lo son tablas y vistas.

SQL, a diferencia de otros lenguajes de programación es un **lenguaje declarativo**; cuando se escribe una instrucción SQL, se describe lo que debe hacerse, pero no exactamente cómo hacerlo; los detalles exactos de cómo se ejecuta la consulta son manejados internamente por el RDBMS que se está utilizando.

El lenguaje SQL está compuesto por **comandos, cláusulas, operadores y funciones de agregado**. Estos elementos se combinan en las instrucciones para crear, actualizar y manipular las bases de datos.

Las sentencias SQL se agrupan en diferentes sub-sets, de acuerdo a su propósito. Los principales son los siguientes.

- **DDL (Data Definition Language o DDL)**: es el conjunto de sentencias de SQL que se usa para crear y modificar objetos de base de datos .

- **DML (Data Manipulation Language o DML):** es el conjunto de sentencias SQL que se utilizan para consultar los datos dentro de una base de datos.
- **DCL (Data Control Language o DCL):** es el conjunto de sentencias SQL que permiten a un administrador de bases de datos, gestionar y controlar el acceso a los objetos de la misma.

En las próximas lecciones, profundizaremos sobre las sentencias básicas de los subsets DDL y DML, que son los más utilizados.

¿Qué es un modelo de datos y cómo recuperar datos de una tabla?

Un modelo de datos es una **representación conceptual de cómo se organizan y estructuran los datos en una base de datos**. Define las entidades (como las tablas), los atributos (columnas) y las relaciones entre entidades. Los modelos de datos ayudan a garantizar la integridad y coherencia de los datos, y proporcionan una base para el diseño y desarrollo de bases de datos.

Para leer un modelo de datos en SQL, normalmente se examinan la estructura y las relaciones de las tablas. SQL es un lenguaje de programación diseñado para gestionar bases de datos relacionales, y proporciona varios comandos y sentencias para interactuar con el modelo de datos. A continuación se indican algunas sentencias SQL utilizadas habitualmente para leer y comprender un modelo de datos:

- **Recuperación de la estructura de una tabla:** Puede utilizar la sentencia DESCRIBE o SHOW COLUMNS para ver la estructura de una tabla específica. Por ejemplo:

```
DESCRIBE your_name_table;
```

- **Visualización de todas las tablas:** Puede consultar los metadatos de la base de datos para obtener una lista de todas las tablas presentes en el modelo de datos. La sintaxis específica puede variar en función del sistema de base de datos que esté utilizando. Por ejemplo, en MySQL:

```
SHOW TABLES;
```

- **Exploración de relaciones:** Si su modelo de datos incluye relaciones de clave foránea, puede utilizar la cláusula JOIN para combinar tablas relacionadas y recuperar datos en varias tablas. Por ejemplo:

```
SELECT *  
  
FROM table1  
  
JOIN table2 ON table1.id = table2.table1_id;
```

Integridad referencial. Claves primarias y claves externas

La **integridad referencial** es un concepto de los sistemas de gestión de bases de datos (SGBD) que garantiza la coherencia y exactitud de las relaciones entre las tablas de una base de datos relacional. Establece y aplica un conjunto de reglas para mantener la integridad de los datos cuando se utilizan relaciones de clave foránea.

En una base de datos relacional, las tablas suelen estar relacionadas entre sí mediante claves externas. Una clave externa es un campo de una tabla que hace referencia a la clave primaria de otra tabla. La clave primaria identifica de forma exclusiva cada registro de una tabla, mientras que la clave externa establece un vínculo entre tablas haciendo referencia a la clave primaria de otra tabla.

La integridad referencial garantiza que estas relaciones sean válidas y coherentes.

Aplica las siguientes reglas:

1. **Existencia de clave primaria:** Una clave externa en una tabla debe hacer referencia a un valor de clave primaria válido en otra tabla. Esto significa que el valor de clave primaria referenciado debe existir en la tabla referenciada.

2. **Ausencia de registros huérfanos:** Si una clave externa hace referencia a una clave primaria, el registro de referencia no debe eliminarse ni modificarse de forma que el valor de la clave externa deje de ser válido. En otras palabras, no se puede eliminar un registro de una tabla referenciada si tiene registros dependientes en otra tabla.
3. **Actualizaciones y eliminaciones en cascada:** Cuando se actualiza o elimina un registro referenciado, los cambios pueden propagarse a los registros relacionados automáticamente. Esto se conoce como actualizaciones o eliminaciones en cascada, en las que los cambios se propagan en cascada a través de la relación para mantener la integridad.

Al aplicar la integridad referencial, el SGBD garantiza que los datos sean coherentes y válidos en todas las tablas. Ayuda a evitar situaciones en las que un registro hace referencia a otro que no existe o en las que se violan las dependencias referenciales. Las violaciones de la integridad referencial pueden provocar incoherencias y anomalías en los datos.

Las reglas de integridad referencial pueden definirse y aplicarse a nivel de esquema de base de datos mediante restricciones. La restricción más común utilizada para la integridad referencial es la restricción de clave externa, que especifica las relaciones entre tablas y las acciones que deben realizarse cuando se realizan cambios en los registros referenciados (por ejemplo, CASCADE, SET NULL, SET DEFAULT).

En general, la integridad referencial desempeña un papel crucial en el mantenimiento de la exactitud y fiabilidad de los datos en una base de datos relacional, garantizando que las relaciones entre tablas se definen y mantienen correctamente.

Claves primarias y externas

En SQL, las **claves primarias** son columnas o combinaciones de columnas que identifican de forma única cada registro en una tabla.

Una **clave externa** es una columna o una combinación de columnas de una tabla que establece un vínculo o relación entre dos tablas. Representa una referencia a la clave primaria de otra tabla, creando una relación entre las dos tablas.

El propósito de una clave externa es reforzar la integridad referencial y mantener las relaciones entre tablas relacionadas. Garantiza que los valores de la(s)

columna(s) de clave externa de una tabla se correspondan con los valores de la(s) columna(s) de clave primaria de otra tabla. El uso de claves externas permite establecer relaciones entre tablas y aplicar restricciones a los datos.

Veamos a continuación un ejemplo que ilustra el concepto de clave externa: 📌

Considera **dos tablas**, "**Pedidos**" y "**Cientes**":

- La tabla "Pedidos" contiene información sobre los pedidos, incluido el cliente que realizó el pedido.
- La tabla "Clientes" contiene información sobre los clientes.

```
CREATE TABLE Customers (  
  customer_id INT PRIMARY  
  KEY, customer_name  
  VARCHAR(50)  
  
);  
  
CREATE TABLE Orders (  
  order_id INT PRIMARY KEY,  
  
  order_date  
  DATE,  
  customer_id  
  INT,  
  
  FOREIGN KEY (customer_id) REFERENCES  
  Customers(customer_id)  
  
);
```

En este ejemplo, la columna "**customer_id**" de la tabla "**Pedidos**" es una clave externa que hace referencia a la columna "**customer_id**" de la tabla "**Cientes**". Esto establece una relación entre las dos tablas, en la que cada pedido se asocia a un cliente concreto.

Al definir esta restricción de clave externa, el sistema de gestión de bases de datos garantiza que cualquier valor de la columna "**customer_id**" de la tabla "**Pedidos**" debe existir en la columna "**customer_id**" de la tabla "**Cientes**". Esto ayuda a mantener la integridad de los datos y evita incoherencias o registros

huérfanos en la base de datos.

Las claves foráneas pueden utilizarse para realizar uniones entre tablas, recuperar datos relacionados y aplicar restricciones de integridad de datos. Son un concepto fundamental en el diseño de bases de datos y se utilizan ampliamente en bases de datos relacionales.

Querys anidadas ↓

Las consultas anidadas, también conocidas como subconsultas, son consultas que **están incrustadas dentro de otra consulta**. Permiten utilizar el resultado de una consulta como entrada para otra consulta. Las consultas anidadas pueden utilizarse en varias partes de una sentencia SQL, como SELECT, FROM, WHERE y HAVING. A continuación se muestran algunos ejemplos de consultas anidadas

Subconsulta en WHERE:

```
SELECT      column1,  
column2      FROM  
table1  
  
WHERE column1 IN (SELECT column1 FROM table2);
```

En este ejemplo, la subconsulta (SELECT columna1 FROM tabla2) recupera una lista de valores de la tabla2. A continuación, la consulta externa selecciona las filas de la tabla1 en las que la columna1 coincide con cualquier valor del resultado de la subconsulta.

Este es sólo un ejemplo de cómo pueden utilizarse las consultas anidadas en SQL. Las subconsultas constituyen una potente herramienta para realizar manipulaciones complejas de datos y hacer que las consultas sean más flexibles y dinámicas. Es importante conocer la sintaxis y las reglas específicas del sistema de gestión de bases de datos que se utilice, ya que puede haber ligeras variaciones en la forma de escribir y ejecutar las subconsultas.

Consultas de selección con funciones de agrupación

Las consultas de selección con funciones de agrupación en bases de datos relacionales **permiten realizar cálculos y análisis de datos agregados sobre grupos de registros**. Estas funciones de agrupación permiten resumir y extraer información útil de la base de datos.

Algunas de las funciones de agrupación más comunes son:

- **COUNT:** Esta función cuenta el número de registros que cumplen cierta condición en un grupo. Por ejemplo, se puede utilizar para contar el número de clientes en cada ciudad.
- **SUM:** Calcula la suma de los valores de una columna en un grupo. Por ejemplo, se puede utilizar para obtener la suma total de ventas por categoría de productos.
- **AVG:** Calcula el promedio de los valores de una columna en un grupo. Por ejemplo, se puede utilizar para obtener el promedio de calificaciones de los estudiantes por asignatura.
- **MAX:** Devuelve el valor máximo de una columna en un grupo. Por ejemplo, se puede utilizar para obtener la edad máxima de los empleados en cada departamento.
- **MIN:** Devuelve el valor mínimo de una columna en un grupo. Por ejemplo, se puede utilizar para obtener el salario mínimo de los empleados en cada categoría de trabajo.

Estas funciones de agrupación se utilizan en combinación con la cláusula GROUP BY en consultas SQL. La cláusula GROUP BY divide los registros en grupos basados en uno o más campos y luego se aplican las funciones de agrupación a cada grupo. Esto permite realizar cálculos agregados y resumir los datos de manera significativa.

Aquí hay un ejemplo de una consulta de selección con funciones de agrupación:

```
SELECT category, COUNT(*) as total_products, AVG(price) as  
avg_price, MAX(price) as max_price  
  
FROM products  
GROUP BY category;
```

En este ejemplo, se agrupan los productos por categoría y se calcula el número total de productos, el precio promedio y el precio máximo para cada categoría.

Las consultas de selección con funciones de agrupación son muy útiles para obtener información resumida y estadísticas sobre conjuntos de datos en bases de datos relacionales. Permiten realizar análisis de datos agregados y tomar decisiones basadas en estos resultados.

Querys con distintos tipos de JOIN (INNER, LEFT, OUTER) ↓

En SQL, un **JOIN** es una **operación relacional que combina filas de dos o más tablas basándose en una columna relacionada entre ellas**. Permite recuperar datos de varias tablas especificando cómo están relacionadas.

Los tipos de JOIN se utilizan normalmente cuando los datos que necesita recuperar están distribuidos en varias tablas y deben consolidarse en un único conjunto de resultados. Al especificar la condición de unión, puede establecer la relación entre las tablas y recuperar los datos que cumplan los criterios especificados.

El tipo de unión más común en SQL es el INNER JOIN, que devuelve sólo las filas que tienen valores coincidentes en ambas tablas basándose en la condición de unión especificada.

A continuación se muestra un ejemplo de INNER JOIN: 📌

```
SELECT Customers.CustomerName,  
Orders.OrderID FROM Customers  
  
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

En este ejemplo, el resultado contendrá el nombre del cliente de la tabla `Clientes` y el correspondiente ID de pedido de la tabla `Pedidos` para las filas coincidentes.

Aparte de **INNER JOIN**, otros tipos de uniones en **SQL** incluyen:

- **LEFT JOIN:** Recupera todas las filas de la tabla izquierda (primera) y las filas coincidentes de la tabla derecha (segunda) basándose en la condición de unión. Si no hay ninguna coincidencia en la tabla derecha, se devuelven valores NULL.
- **RIGHT JOIN:** Recupera todas las filas de la tabla derecha (segunda) y las filas coincidentes de la tabla izquierda (primera) según la condición de unión. Si no hay coincidencias en la tabla izquierda, se devuelven valores NULL.
- **FULL OUTER JOIN:** Recupera todas las filas de ambas tablas e incluye las filas no coincidentes de ambas tablas. Si no hay coincidencias, se devuelven valores NULL para las columnas de la tabla no coincidente.

Estas operaciones de unión permiten combinar datos de varias tablas y obtener un resultado consolidado basado en las relaciones especificada

Cierre

Durante esta lección hemos aprendido a definir una estructura de datos a través de un modelo, a realizar consultas utilizando el lenguaje de consulta estructurado (SQL), a establecer relaciones entre las tablas utilizando claves primarias y foráneas y por último definimos qué son las consultas anidadas y cómo llevarlas a la práctica.

Referencias

- Database Journal. (n.d.). What is MySQL?
<https://www.databasejournal.com/mysql/what-is-mysql/>
- Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). Database systems: The complete book (2nd ed.). Pearson.
- Kroenke, D. M., & Auer, D. J. (2014). Database concepts (7th ed.). Pearson.
- Microsoft. (n.d.). SQL Server documentation.
<https://docs.microsoft.com/en-us/sql/>
- Mode Analytics. (n.d.). SQL tutorial. <https://mode.com/sql-tutorial/>
- SQLCourse.com. (n.d.). SQL tutorial – Beginner & advancer.
<https://www.sqlcourse.com/>

¡Muchas gracias!

Nos vemos en la próxima lección

